

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guión.

Lenguajes de guión de uso general

El modelo de documento web (DOM)

Gestión de eventos

Gestión de errores

El modelo de documento web (DOM)

El Modelo de Objeto del Documento (DOM) es una representación estructurada de un documento web (HTML o XML). Este modelo permite manipular dinámicamente la estructura, el contenido y el estilo de la página web mediante lenguajes de guión como JavaScript.

Estructura de documento

Un documento web se organiza como un árbol de nodos. Cada elemento HTML, texto, atributo o comentario es un nodo dentro del árbol.

Ejemplo básico:

```
<!DOCTYPE html>
<html>
<head>
  <title>Ejemplo DOM</title>
</head>
<body>
  <h1 id="titulo">Hola, DOM</h1>
  <p class="texto">Manipulando documentos con JavaScript.</p>
</body>
</html>
```

Cómo representarlo:

- <html> es el nodo raíz.
- <head> y <body> son nodos hijos.
- Dentro de <body> hay un <h1> y un <p> como nodos hijos.

Navegación por las propiedades de un documento

JavaScript permite acceder y manipular elementos del DOM a través de métodos.

Ejemplo práctico:

JavaScript

```
// Acceder al título mediante su id
let titulo = document.getElementById("titulo");
console.log(titulo.textContent); // "Hola, DOM"

// Acceder al párrafo mediante su clase
let parrafo = document.querySelector(".texto");
console.log(parrafo.textContent); // "Manipulando documentos con JavaScript."
```

Cambio de propiedades mediante lenguajes de guión

JavaScript permite acceder y manipular elementos del DOM a través de métodos.

Ejemplo:

JavaScript

```
// Cambiar el texto del título
titulo.textContent = "¡Título actualizado!";

// Cambiar el color del párrafo
parrafo.style.color = "blue";
```

Tabla con algunas de las propiedades más comunes que puedes manipular con JavaScript para cambiar elementos del DOM:

Propiedad	Descripción	Ejemplo
textContent	Cambia o devuelve el texto contenido dentro de un elemento.	<code>element.textContent = "Nuevo texto";</code>
innerHTML	Cambia o devuelve el contenido HTML dentro de un elemento.	<code>element.innerHTML = "Texto en negrita";</code>
style	Accede a las propiedades CSS en línea del elemento.	<code>element.style.color = "red";</code>
className	Cambia todas las clases de un elemento como una cadena de texto.	<code>element.className = "nueva-clase";</code>
classList	Proporciona métodos para manipular clases, como añadir, eliminar o alternar clases.	<code>element.classList.add("nueva-clase");</code> <code>element.classList.remove("clase-vieja");</code>
id	Cambia o devuelve el atributo id de un elemento.	<code>element.id = "nuevo-id";</code>
src	Cambia o devuelve la URL de una imagen o recurso (usado en , <script>, etc.).	<code>imagen.src = "nueva-imagen.jpg";</code>
href	Cambia o devuelve la URL en un enlace (<a>).	<code>enlace.href = "https://example.com";</code>

Propiedad	Descripción	Ejemplo
value	Cambia o devuelve el valor de un campo de entrada, como <input> o <textarea>.	<code>campo.value = "Nuevo valor";</code>
placeholder	Cambia o devuelve el texto del atributo placeholder en campos de entrada.	<code>campo.placeholder = "Introduce tu nombre";</code>
checked	Cambia o devuelve el estado marcado de un elemento <input type="checkbox"> o <input type="radio">.	<code>checkbox.checked = true;</code>
disabled	Cambia o devuelve si un elemento de formulario está deshabilitado.	<code>boton.disabled = true;</code>
hidden	Cambia o devuelve si un elemento está oculto.	<code>element.hidden = true;</code>
title	Cambia o devuelve el texto del atributo title (tooltip).	<code>element.title = "Información adicional";</code>
tabIndex	Cambia o devuelve el orden de tabulación del elemento.	<code>element.tabIndex = 1;</code>
alt	Cambia o devuelve el texto alternativo de una imagen ().	<code>imagen.alt = "Descripción de la imagen";</code>
width / height	Cambia o devuelve el ancho y alto de elementos como imágenes o videos.	<code>imagen.width = 300; imagen.height = 200;</code>

Propiedad	Descripción	Ejemplo
outerHTML	Cambia o devuelve el HTML completo del elemento, incluyendo el propio elemento.	<code>element.outerHTML = "<div>Nuevo contenido</div>";</code>
dataset	Proporciona acceso a los atributos personalizados data- [*] .	<code>element.dataset.valor = "nuevo-valor";</code>
attributes	Devuelve un array con todos los atributos del elemento.	<code>console.log(element.attributes);</code>

Gestión de eventos

Tipos de eventos

Un evento es una interacción del usuario o del navegador con la página web. Los eventos comunes incluyen:

- **Clic:** click
- **Paso del ratón:** mouseover
- **Entrada de texto:** input
- **Carga del documento:** load

Ejemplo básico:

html

```
<button id="miBoton">Haz clic</button>
<script>
  // Escuchar el evento click
  let boton = document.getElementById("miBoton");
  boton.addEventListener("click", function () {
    alert("¡Botón clicado!");
  });
</script>
```

Tabla con los principales **eventos nativos en JavaScript**, una descripción de lo que hacen y un ejemplo de uso:

Evento	Descripción	Ejemplo de Uso
click	Se dispara cuando el usuario hace clic en un elemento.	<code>element.addEventListener('click', () => { console.log("Elemento clicado"); });</code>
dblclick	Se dispara cuando el usuario hace doble clic en un elemento.	<code>element.addEventListener('dblclick', () => { console.log("Doble clic detectado"); });</code>
mouseenter	Se dispara cuando el puntero entra en un elemento.	<code>element.addEventListener('mouseenter', () => { console.log("Mouse entró al elemento"); });</code>
mouseleave	Se dispara cuando el puntero sale de un elemento.	<code>element.addEventListener('mouseleave', () => { console.log("Mouse salió del elemento"); });</code>
keydown	Se dispara cuando el usuario presiona una tecla en el teclado.	<code>document.addEventListener('keydown', (e) => { console.log("Tecla presionada: " + e.key); });</code>
keyup	Se dispara cuando el usuario suelta una tecla.	<code>document.addEventListener('keyup', (e) => { console.log("Tecla liberada: " + e.key); });</code>
submit	Se dispara cuando se envía un formulario.	<code>form.addEventListener('submit', (e) => { e.preventDefault(); console.log("Formulario enviado"); });</code>
change	Se dispara cuando se cambia el valor de un campo de entrada, como un <code><input></code> o <code><select></code> .	<code>input.addEventListener('change', () => { console.log("Valor cambiado"); });</code>
input	Se dispara mientras el usuario escribe en un campo de entrada.	<code>input.addEventListener('input', () => { console.log("Entrada detectada: " + input.value); });</code>

Evento	Descripción	Ejemplo de Uso
focus	Se dispara cuando un elemento recibe el foco (por ejemplo, al hacer clic en un campo de texto).	<pre>input.addEventListener('focus', () => { console.log("Campo enfocado"); });</pre>
blur	Se dispara cuando un elemento pierde el foco.	<pre>input.addEventListener('blur', () => { console.log("Campo desenfocado"); });</pre>
load	Se dispara cuando la página o un recurso como una imagen termina de cargarse.	<pre>window.addEventListener('load', () => { console.log("Página completamente cargada"); });</pre>
resize	Se dispara cuando se cambia el tamaño de la ventana del navegador.	<pre>window.addEventListener('resize', () => { console.log("Ventana redimensionada"); });</pre>
scroll	Se dispara cuando se desplaza el contenido de un elemento o de la ventana.	<pre>window.addEventListener('scroll', () => { console.log("Scroll detectado"); });</pre>
contextmenu	Se dispara al hacer clic derecho, abriendo el menú contextual.	<pre>element.addEventListener('contextmenu', (e) => { e.preventDefault(); console.log("Menú contextual bloqueado"); });</pre>
dragstart	Se dispara cuando se comienza a arrastrar un elemento.	<pre>element.addEventListener('dragstart', () => { console.log("Inicio de arrastre"); });</pre>
dragover	Se dispara mientras se arrastra un elemento sobre una zona válida de destino.	<pre>element.addEventListener('dragover', (e) => { e.preventDefault(); console.log("Elemento arrastrado sobre una zona válida"); });</pre>
drop	Se dispara cuando un elemento arrastrado es soltado en una zona válida de destino.	<pre>element.addEventListener('drop', (e) => { e.preventDefault(); console.log("Elemento soltado en la zona de destino"); });</pre>

Evento	Descripción	Ejemplo de Uso
error	Se dispara cuando ocurre un error de carga en un recurso como una imagen o un script.	<code>img.addEventListener('error', () => { console.log("Error al cargar la imagen"); });</code>
animationstart	Se dispara al iniciar una animación CSS en un elemento.	<code>element.addEventListener('animationstart', () => { console.log("Animación iniciada"); });</code>
animationend	Se dispara al finalizar una animación CSS en un elemento.	<code>element.addEventListener('animationend', () => { console.log("Animación terminada"); });</code>
transitionend	Se dispara al finalizar una transición CSS en un elemento.	<code>element.addEventListener('transitionend', () => { console.log("Transición completada"); });</code>

Técnicas para el manejo de eventos

- **Manejadores en línea:** Definir eventos directamente en el HTML.

html:

```
<button onclick="alert('¡Botón clicado!')">Haz clic</button>
```

- **Manejadores mediante script:** Usar `addEventListener` para mayor flexibilidad.

javascript:

```
boton.addEventListener("click", function () {  
    alert("¡Botón clicado!");  
});
```

Eventos de carga inicial

El evento `DOMContentLoaded` asegura que el script se ejecute cuando el DOM esté completamente cargado.

Ejemplo:

javascript

```
document.addEventListener("DOMContentLoaded", function () {  
    console.log("¡El documento está listo para ser manipulado!");  
});
```

Delegación y propagación de eventos

La **delegación de eventos** consiste en asignar un evento a un elemento padre en lugar de a cada hijo individual. Esto mejora el rendimiento cuando hay muchos elementos.

Ejemplo:

html

```
<ul id="lista">
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>
<script>
  document.getElementById("lista").addEventListener("click", function (e) {
    if (e.target.tagName === "LI") {
      alert(`Has hecho clic en: ${e.target.textContent}`);
    }
  });
</script>
```

La **propagación** es cómo los eventos viajan desde el elemento más interno (foco) al más externo (raíz). Puedes evitarlo usando `e.stopPropagation()`.

Gestión de errores

Manejo de error «No lenguajes de guión habilitados» (noscript)

Algunos navegadores desactivan JavaScript. Puedes mostrar un mensaje alternativo usando <noscript>.

Ejemplo:

html

```
<noscript>  
  Tu navegador tiene JavaScript desactivado. Por favor, actívalo para usar esta página.  
</noscript>
```

Chequeo de errores en funciones

Puedes validar entradas para evitar errores en tiempo de ejecución.

Ejemplo:

javascript

```
function dividir(a, b) {  
    if (b === 0) {  
        throw new Error("No se puede dividir entre cero.");  
    }  
    return a / b;  
}  
  
try {  
    console.log(dividir(10, 2)); // 5  
    console.log(dividir(10, 0)); // Error  
} catch (error) {  
    console.error(error.message);  
}
```

Captura de errores

El bloque try-catch se usa para manejar errores.

Ejemplo práctico:

javascript

```
try {  
    let numero = parseInt(prompt("Introduce un número:"));  
    if (isNaN(numero)) {  
        throw new Error("Debes introducir un número válido.");  
    }  
    console.log(`El número es: ${numero}`);  
} catch (error) {  
    alert(error.message);  
}
```

Uso de puntos de ruptura

Los puntos de ruptura son herramientas de depuración que detienen la ejecución del script para inspeccionar el estado del programa. Se configuran en las herramientas de desarrollo del navegador.

Cómo usarlos:

1. Abre la consola del navegador (Ctrl+Shift+I o F12).
2. Ve a la pestaña **Sources**.
3. Añade un punto de ruptura haciendo clic en la línea deseada del código.

Ejercicio: Interacción con un botón y una caja de texto

Descripción:

En el repositorio **estaticos** crear una carpeta en **ejercicios-uf2** llamada **ejercicio-3**.

En ella Crea una página web (index.html) que tenga:

1. Un título que diga: “Haz clic en el botón para cambiarme”.
2. Un párrafo que inicialmente esté oculto.
3. Un botón que, al hacer clic:
 - Cambie el texto del título.
 - Muestre el párrafo.
 - Cambie el color de fondo del botón.
4. Una entrada de texto donde el usuario pueda escribir un nuevo título, y al presionar “Enter”, este cambie el texto del título.

Tareas para resolver:

1. **Cambiar propiedades del botón:** ¿Qué sucede si cambias el color del texto en lugar del fondo? ¿Cómo lo harías?
2. **Validación:** Evita que el usuario ingrese un título vacío en el campo de texto. Muestra una alerta si lo intenta.
3. **Experimento:** Agrega un nuevo botón que, al hacer clic, oculte nuevamente el párrafo y restaure los colores originales del botón.

¿Te animas a resolver las tareas? 😊